

Introducció a les aplicacions web



1. Tipus d'aplicacions

Quan estem utilitzant un ordinador, una tauleta o un telèfon mòbil, quins tipus d'aplicacions o programes podem estar utilitzant? Bàsicament distingim dos grans grups:

- **Aplicacions sense connexió a Internet.** Este tipus d'aplicacions no necessiten cap connexió a Internet o a una xarxa local d'ordinadors per a funcionar. Solen dir-se **aplicacions d'escriptori**, i podem trobar exemples molt variats: un processador de textos, un lector de llibres electrònics, un reproductor de música o vídeo, i fins i tot videojocs que tinguem instal·lats.
- **Aplicacions amb connexió a Internet.** Estes aplicacions sí que necessiten connexió, ja siga a Internet o a una xarxa local. Dins d'este grup, trobem diversos subtipus:
 - **Aplicacions P2P** (*peer-to-peer*): tots els elements connectats a la xarxa tenen el mateix "rang" i compartixen informació entre ells. És el mecanisme en el qual es basen diversos programes de descàrrega, com els d'arxius tipus *torrent.
 - **Aplicacions client-servidor:** consistixen en el fet que un conjunt d'ordinadors (anomenats clients) es connecten a un central (anomenat servidor), que proporciona la informació i els servicis sol·licitats. És el cas dels videojocs en línia, on l'ordinador del jugador (client) es connecta a un servidor central que gestiona el joc. Dins de les aplicacions client-servidor, hi ha un subtipus especialment nombrós, les **aplicacions web**. És en este subtipus en el qual ens centrarem.

1.1. Què és una aplicació web?

Una aplicació web és un tipus de programari que s'executa en un servidor i s'accedix a través d'un navegador web. El navegador actua com a interfície entre l'usuari i el servidor, on es processa la lògica de negoci, s'accedix a bases de dades i es genera la resposta que el navegador presenta a l'usuari.

Amb l'avanç de les tecnologies web, el concepte d'aplicació web s'ha ampliat. Actualment, es considera **aplicació web** a qualsevol programari disenrotllat amb tecnologies com a HTML, CSS, JavaScript, PHP, etc., que s'executa en un navegador o en un entorn que l'emula, com un *WebView* o un *motor de navegador embegut*.

WebView és un component que permet incrustar contingut web dins d'una aplicació mòbil nativa. És comú en aplicacions com Facebook o Instagram, on es carreguen pàgines externes sense obrir el navegador del sistema. Encara que el seu ús és més controlat, ofereix rapidesa i cohesió dins de l'app.

1.1.2 Tipus d'aplicacions web modernes

Hui dia, podem identificar diversos tipus d'aplicacions web modernes segons el seu entorn d'execució i les seues capacitats:

- **Aplicacions híbrides (basades en WebView):** són aplicacions desenvolupades amb tecnologies web i empaquetades com a aplicacions mòbils usant ferramentes com *Ionic* o *Capacitor*. S'executen dins d'un WebView i gràcies a plugins específics permeten accedir a funcionalitats del dispositiu com la càmera, GPS, notifiacions, etc.
- **Aplicacions d'escriptori realitzades amb tecnologies web:** utilitzen frameworks com *Electron* o *Tauri* per a crear aplicacions d'escriptori que funcionen com a programes nadius en Windows, macOS o Linux. Combinen un motor web (com Chromium) amb accés al sistema operatiu, permetent construir interfícies amb tecnologies web reutilitzant el mateix codi base del frontend. Encara que permeten accés al sistema d'arxius i notifiacions, solen consumir més recursos que una aplicació nativa tradicional. Un exemple representatiu és Visual Studio Code, desenvolupat amb Electron.
- **Progressive Web Apps (PWA):** són aplicacions web que poden instal·lar-se des del navegador i oferir funcionalitats típiques d'apps natives, com a notifiacions push, accés sense connexió o execució en segon pla. Un exemple destacat és Twitter Lite.

2. Arquitectura d'una aplicació web

2.1. Què és "la web"?

La web és una plataforma global que permet accedir a una gran varietat de continguts i servicis: pàgines informatives, aplicacions interactives, xarxes socials, fòrums, vídeos, videojocs, etc.

Des dels seus inicis en els anys 90, amb pàgines estàtiques i correu electrònic, ha evolucionat fins a convertir-se en una plataforma complexa i dinàmica. L'anomenada **Web 2.0** va introduir la participació de l'usuari (blogs, xarxes socials, wikis), i hui dia parlem fins i tot d'una **Web 3.0**, més centrada en la descentralització, la intel·ligència artificial i la personalització.

2.2. Elements d'una aplicació web

Una aplicació web moderna sol estar composta per **tres elements principals** que treballen de manera coordinada:

- **Front-end (client):** és la part de l'aplicació que **s'executa en el navegador de l'usuari** i s'encarrega de mostrar la interfície visual, capturar la interacció (clics, formularis, navegació), i enviar/rebre dades des del servidor. Treballa directament amb l'experiència de l'usuari.
- **Tecnologies habituals:**
 - Llenguatges: HTML, CSS, JavaScript.
 - Frameworks/llibreries modernes: React, Angular, Vue.
 - Ferramentes complementàries:
 - Millora d'estils: Sass (preprocessador), Bootstrap, Tailwind.

- Millora del codi JavaScript: TypeScript (superset amb tipat estàtic).
- **Back-end (servidor):** és el component que **s'executa en un servidor remot** i gestiona la lògica de negoci, l'accés a dades, l'autenticació d'usuaris i la resposta a les peticions del client. És el “motor” de l'aplicació.
- **Tecnologies habituals:**
 - Llenguatges: PHP, Node.js, Python (Django/Flask), Java (Spring), etc.
 - Bases de dades: MySQL, PostgreSQL, MongoDB, Redis.
 - Servidors: Apache, Nginx, Express.
 - Ferramentes útils: ORMs com a Prisma o Sequelize.
- **Comunicació entre client i servidor:** la interacció es realitza mitjançant **peticions HTTP/HTTPS**, utilitzant normalment APIs (com REST o GraphQL). La informació sol enviar-se en format JSON, encara que poden usar-se altres formats com XML o HTML, depenent del context.

2.3. Funcionament d'una aplicació web

Les aplicacions web segueixen una **arquitectura client-servidor**, en la qual les tasques es repartixen entre:

- El **client** (navegador), que sol·licita recursos o servicis.
- El **servidor**, que processa eixes sol·licituds, accedix a les dades si és necessari, i retorna la resposta corresponent.

Seqüència bàsica de funcionament

1. L'usuari accedix a l'aplicació des del seu navegador (client)
2. El client envia una petició al servidor (per exemple, per a iniciar sessió o consultar informació)
3. El servidor processa la sol·licitud, accedix a la base de dades si és necessari i genera una resposta.
4. El servidor envia la resposta (dades en format JSON, HTML, etc.)
5. El client interpreta la resposta i mostra la informació a l'usuari i, segons el cas, realitza noves peticions o finalitza la sessió.

2.4 Arquitectura en capes

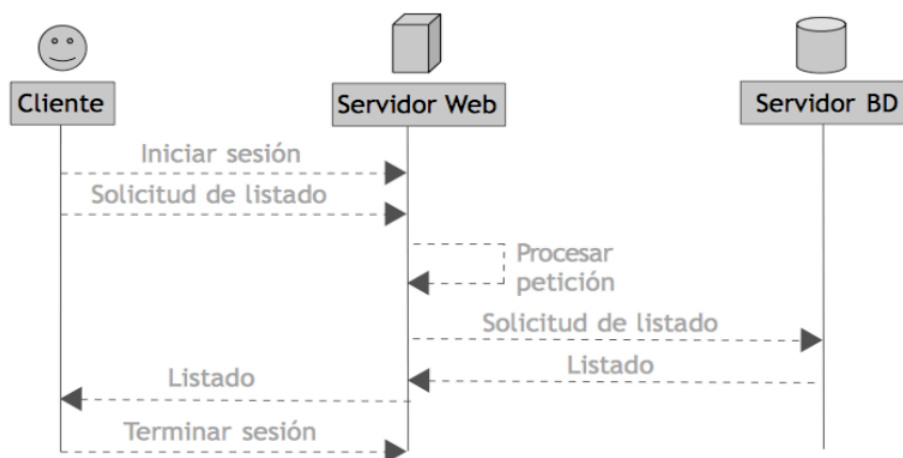
En moltes aplicacions web modernes, el servidor no és un únic sistema. En el seu lloc, s'organitza en una **arquitectura en capes** (també anomenada *multi-tier*), on diferents servicis o màquines s'especialitzen en funcions concretes. Això millora el **rendiment**, la **escalabilitat** i la **seguretat** del sistema.

Per exemple:

- Una màquina pot encarregar-se del **servidor web**, que atén les peticions del client.
- Una altra, del **servidor de bases de dades**, que emmagatzema i gestiona la informació.
- Altres servicis addicionals poden gestionar el **correu electrònic**, la **autenticació** o **emmagatzematge en el núvol**.

Exemple pràctic

Suposem que un usuari accedix a una aplicació web per a consultar notícies. Este procés pot ser representat amb un diagrama de seqüència com el següent:



1. El client (navegador) envia una sol·licitud d'inici de sessió al servidor web.
2. Després d'iniciar sessió correctament, el client sol·licita un llistat de notícies.
3. El servidor web processa la petició i, si necessita dades, consulta al servidor de bases de dades.
4. El servidor de bases de dades respon amb les dades sol·licitades al servidor web.
5. El servidor web genera una resposta (per exemple, un JSON o un HTML) i l'envia al client.
6. El client mostra les dades i permet noves interaccions o tancar sessió.

2.4.1 Tipus d'arquitectura

En funció de com s'organitzen els elements del sistema, podem distingir dos models principals:

- **Arquitectura de dos nivells (2-tier)**, tant el servidor web com el servidor de bases de dades s'executen en la mateixa màquina. És una solució senzilla i econòmica, adequada per a projectes xicotets, entorns de desenvolupament o proves. No obstant això, presenta limitacions importants en termes de seguretat, escalabilitat i rendiment, especialment quan el nombre d'usuaris o el volum de dades creix.
- **Arquitectura de tres nivells (3-tier)**, el servidor web i el servidor de bases de dades estan separats físicament, la qual cosa permet distribuir millor la càrrega, millorar el rendiment, i augmentar la seguretat. És el model més comú en entorns empresarials i sistemes en producció.

Més enllà d'estos models clàssics, hui dia també s'empren **altres arquitectures modernes**, com:

- La **arquitectura de microservicis**, on cada funcionalitat (autenticació, pagaments, busca, etc.) s'implementa com un servici independent, la qual cosa permet desplegar, escalar i mantindre cada un per separat.
- La **arquitectura desacoblada (frontend-backend)**, en la qual el frontend (interfície) i el backend (la lògica i dades) es desenvolupen i despleguen per separat, comunicant-se a través d'una API (normalment REST o GraphQL). Això permet més flexibilitat i reutilització entre plataformes (web, mòbil, etc.).
- Les **arquitectures serverless**, proporcionades per proveïdors de servicis *cloud* (AWS, Azure...) que administren l'estructura automàticament, i el programador només implementa funcions més lleugeres.

3. Localització i accés a aplicacions web

Perquè una aplicació web estiga disponible en Internet i accessible des de qualsevol navegador, és necessari tindre en compte tres elements clau:

- **On s'allotja l'aplicació** (hosting o servidor).
- **Com fer accessible l'aplicació amb un nom de domini.**
- **Com s'accedix als recursos** dins de l'aplicació (URLs).

En este apartat s'aborden els conceptes bàsics necessaris per a comprendre com es publica una aplicació web i com s'accedix a ella des de qualsevol part del món. Alguns aspectes més tècnics, com els protocols de comunicació i el sistema DNS, s'estudiaran detalladament en apartats posteriors.

3.1 Allotjament web (hosting)

El **hosting** és el servei que permet emmagatzemar i servir els arxius d'una aplicació web (HTML, CSS, JS, imatges, bases de dades, etc.) des d'un servidor connectat permanentment a Internet.

Existixen diferents opcions d'allotjament segons el tipus de projecte, el nivell tècnic de l'equip i els recursos disponibles:

- **Servidor propi:** implica desplegar l'aplicació en una màquina física gestionada directament per l'empresa o l'equip de desenvolupament. Oferix control total, però requereix coneixements avançats, manteniment continu i una inversió inicial considerable.
- **Hosting tradicional:** empreses com Ionos, Hostinger o OVH ofereixen allotjament web professional amb diferents modalitats, adaptades al tipus de projecte i al nivell tècnic de l'usuari. Les més comunes són:
 - **Allotjament compartit:** diversos llocs web compartixen un mateix servidor físic i els seus recursos. És l'opció més econòmica i senzilla d'usar, ideal per a projectes xicotets, pàgines personals o llocs amb poc trànsit. No requereix coneixements tècnics i sol incloure panells de control fàcils d'usar. No obstant això, en compartir recursos amb altres usuaris, el rendiment pot veure's afectat per l'ús de tercers. Quant a la seguretat, depén en gran manera de la correcta configuració del proveïdor, ja que una mala gestió pot permetre que les fallades d'una web afecten a les altres.
 - **Servidor privat virtual (VPS, Virtual Private Server):** es tracta d'una partició virtual dins d'un servidor físic, que funciona com un servidor independent amb els seus propis recursos assignats. Oferix millor rendiment, més control i opcions de configuració que l'allotjament compartit. És una bona opció per a aplicacions de grandària mitjana o que requereixen ajustos tècnics específics. A nivell de seguretat, proporciona un entorn més aïllat i configurable, la qual cosa permet aplicar polítiques de protecció més avançades, encara que també exigeix coneixements per a administrar-lo correctament.
 - **Servidor dedicat:** consistix a llogar un servidor físic complet, exclusiu per a una única aplicació o client. És l'opció més potent i flexible, recomanada per a projectes que manegen grans volums de trànsit, dades sensibles o servicis crítics. Permet control total sobre el sistema i la seguretat, però també requereix experiència tècnica per a gestionar-lo i mantindre'l adequadament. Encara que el

proveïdor s'encarrega de la infraestructura, la protecció de l'entorn depén de com es configure i administre.

- **Plataformes modernes de desplegament:** servicis en el núvol com AWS, Azure o plataformes especialitzades com Vercel i Netlify, permeten desplegar aplicacions de manera automatitzada, amb integració contínua i escalat dinàmic. Estan especialment orientades a architectures modernes com APIs, microservicis i frontends desacoblats.

3.2 Noms de domini

Encara que les aplicacions web estan allotjades en servidors identificats per la seua adreça IP (per exemple, 172.217.168.78), estos números no són pràctics per a l'ús diari.

Per a facilitar l'accés, s'utilitzen **noms de domini**, que són adreces web llegibles i fàcils de recordar, com *www.iessanvicente.com*. Internament, el nom de domini està associat a la IP real del servidor on s'allotja l'aplicació. Gràcies a esta associació, el navegador pot dirigir correctament les sol·licituds sense que l'usuari conega l'adreça IP.

Per exemple, en accedir a **www.google.es**, el sistema tradueix eixe nom en l'adreça IP corresponent a un dels servidors de Google. Esta conversió la realitza un sistema anomenat **DNS** (*Domain Name System*), que s'explicarà en profunditat.

3.2.1 Registre de domini

Per a disposar d'un domini propi, és necessari realitzar un **registre de domini**, un procés gestionat per empreses autoritzades per la **ICANN** (*Internet Corporation for Assigned Names and Numbers*).

Per a registrar un domini, és necessari:

1. Triar un nom de domini que estiga disponible.
2. Seleccionar una extensió adequada (.com, .és, .org, etc.).
3. Registrar el domini a través d'un **agent registrador** acreditat.
4. Configurar els registres DNS necessaris per a vincular el domini amb el servidor que allotja l'aplicació.

Una vegada registrat i vinculat al servidor mitjançant el sistema DNS, el domini estarà actiu i accessible des d'Internet.

3.3 URLs

Una **URL** (*Uniform Resource Locator*) és l'adreça completa que s'usa per a localitzar un recurs específic dins d'una aplicació web. És la forma en la qual els navegadors realitzen peticions als servidors.

Per exemple, quan escrivim en un navegador una adreça com *http://www.miweb.com/paginas/pagina.html*, estem introduint una URL per a localitzar un recurs (en este cas, una pàgina HTML).

L'estructura general d'una URL és la següent:

`protocol://subdomini.domini.com/carpeta/pagina?param=valor`

- El **protocol**, definix les regles per a la comunicació entre client i servidor. En una URL, apareix a l'inici, abans dels dos punts i les dos barres (//). En l'exemple anterior, el protocol és **http** (*HiperText Transfer Protocol*). Veurem més avant altres protocols.
- El **subdomini**, és opcional i permet organitzar servicis dins d'un mateix domini principal. És habitual que les empreses de hosting permeten registrar subdominis, encara que algunes ho oferisquen amb unes certes restriccions, ja siga pel nombre de subdominis permesos o pel servici que presten. Per exemple, si tenim el domini *www.miweb.com* de l'exemple anterior com a URL principal, podríem tindre a més, *tienda.miweb.com* com a subdomini per a una botiga associada a la web.
- El **nom de domini**, identifica el servidor al qual es desitja accedir. Va just darrere del protocol, fins a la següent barra. Normalment acaba en *.com*, *.és*, *.net*, etc. En el nostre exemple seria *www.miweb.com*
- La **ruta cap al recurs**, indica la ubicació concreta de l'arxiu dins del servidor (*/pàgines/pàgina.HTML*, en l'exemple).
- Uns **paràmetres** opcionals, que proporcionen informació addicional a la sol·licitud. Útil per a generar continguts dinàmics (*?param=valor*).

Este format permet accedir de manera precisa i estructurada a qualsevol recurs dins d'una aplicació web.

Exercici 1:

Hem vist que la ICANN autoritza diverses empreses per a gestionar el registre de noms de domini en Internet. Estes empreses, conegudes com a agents registradors de domini, ofereixen diferents plans de pagament per a este servici. Busca en Internet una llista d'agents registradors de domini oficials per a Espanya i consulta els preus que tenen alguns d'ells.

4. Protocols més utilitzats

Perquè els clients i servidors puguin comunicar-se, han de seguir un **protocol de comunicació**. Un protocol definix una sèrie de regles que especifiquen quin tipus de missatges s'intercanviaran, en quina orde i com ha d'interpretar-los cada part. Això garantix que els dos extrems de la comunicació s'entenguen, fins i tot si són tecnologies distintes.

4.1 Protocol base: TCP/IP

Totes les comunicacions en Internet es recolzen en la **família de protocols TCP/IP**, que es compon de dos elements principals:

- **Protocol TCP (Transmission Control Protocol)**: s'encarrega de dividir la informació en paquets, assegurar el seu lliurament complet i en l'orde correcte.
- **Protocol IP (Internet Protocol)**: identifica els dispositius mitjançant adreces IP i dirigix els paquets entre origen i destinació.

Sobre esta base es construeixen protocols més específics, com els que usen les aplicacions web, el correu electrònic o la transferència d'arxius.

4.2 Protocols web principals: HTTP i HTTPS

En el cas de les aplicacions web, els protocols més utilitzats són:

- **HTTP (HyperText Transfer Protocol)**: és el protocol estàndard des dels anys 90 per a la transferència de pàgines web. Funciona seguint un esquema de petició-resposta, en el qual el client sol·licita un recurs (com un arxiu HTML) i el servidor l'envia com a resposta. Este protocol no és segur, ja que les dades viatgen sense xifrar. La seua evolució ha originat versions posteriors com ara **HTTP/2** o **HTTP/3**, que han millorat significativament els temps de resposta al permetre peticions simultànies.
- **HTTPS (HTTP Secure)**: és la versió segura d'HTTP. Afig una capa de xifratge mitjançant TLS (abans SSL), de manera que les dades transmeses no poden ser interceptats ni manipulats per tercers. És obligatori en aplicacions que manegen informació sensible, com a contrasenyes, pagaments, dades personals o bancàries. Els navegadors moderns prioritzen automàticament l'ús d'HTTPS quan detecten formularis o zones d'autenticació, i mostren un cadenat en la barra de direccions quan la connexió és segura. No obstant això, és necessari que el servidor web estiga correctament configurat per a suportar HTTPS, incloent-hi la instal·lació d'un certificat digital vàlid.

4.3 Altres protocols d'Internet

Encara que no s'usen directament en aplicacions web, existixen altres protocols de comunicació essencials per a servicis en Internet:

- **SMTP (Simple Mail Transfer Protocol)**: s'utilitza per a enviar correus electrònics.
- **POP3 (Post Office Protocol v3) / IMAP (Internet Message Access Protocol)**: s'utilitzen per a rebre correus electrònics, depenent de si es descarreguen o es consulten directament des del servidor.
- **FTP (File Transfer Protocol)**: s'emptra per a transferir arxius entre el client i el servidor. És útil durant el desenvolupament web per a pujar arxius al hosting, encara que cada vegada s'usa menys en favor de mètodes més segurs com SFTP o desplegaments automàtics des de repositoris.

4.4. Funcionament d'HTTP/HTTPS

El protocol HTTP, i la seua versió segura HTTPS, són la base de la comunicació en les aplicacions web. Els dos funcionen sota un model client-servidor, on el navegador (client) realitza una petició i el servidor respon amb el contingut sol·licitat o un missatge d'error. Esta comunicació es realitza seguint un format estandarditzat, que permet que els dos extrems puguin interpretar-la correctament.

Peticions HTTP (requests)

Una petició HTTP és el missatge que el client envia al servidor per a sol·licitar un recurs. Sol estar composta per:

- La **URL** del recurs sol·licitat.
- Les **capçaleres de petició**, que inclouen informació sobre el navegador, l'idioma de l'usuari, el tipus de contingut acceptat, etc.
- El **cos** (opcional), que pot contindre dades com els valors d'un formulari o arxius que es volen pujar.

Internament, esta informació s'encapsula en paquets que es fragmenten i envien seguint les regles del protocol TCP.

Respostes HTTP (response)

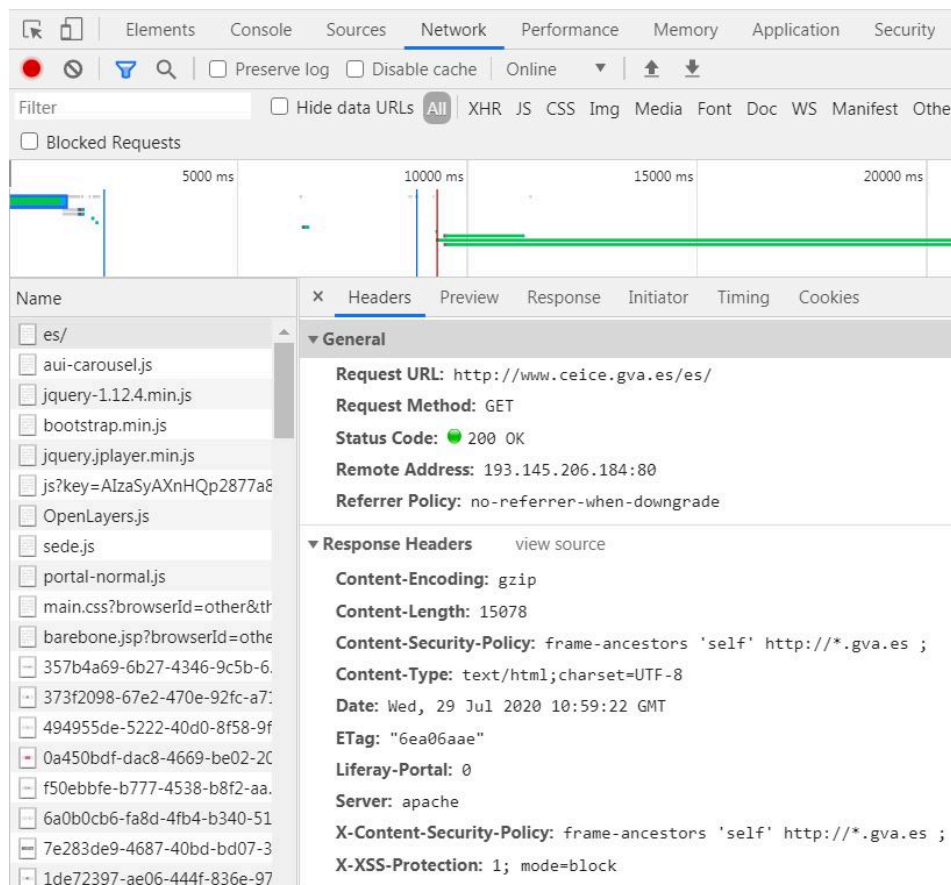
El servidor analitza la petició i respon amb un missatge estructurat, que inclou:

- Un **codi d'estat**, que indica el resultat de la petició. Estos codis s'agrupen en diverses categories:
 - Els codis 2xx indiquen una resposta satisfactòria (*200 OK*).
 - Els codis 3xx indiquen que hi ha hagut algun tipus de redirecció (*301 Moveu Permanently*, *302 Found*).
 - Els codis 4xx indiquen un error per part del client. (*404 Not Found*, *403 Forbidden*).
 - Els codis 5xx indiquen un error per part del servidor (*500 Internal Server Error*, *503 Service Unavailable*).
- Unes **capçaleres de resposta**, que detallen aspectes com el tipus de contingut retornat (tipus MIME), la grandària del recurs o la data d'última modificació.
- El **contingut** pròpiament dit, que pot ser una pàgina HTML, un arxiu, una imatge o fins i tot un missatge d'error en format web.

4.5 Anàlisi de trànsit HTTP amb Google Chrome

Per a observar este intercanvi en temps real, es pot usar l'**eina de desenvolupament** de Google Chrome, en la pestanya **Network**, per a analitzar qualsevol comunicació entre el navegador i un servidor web. Això permet identificar codis d'estat, capçaleres i contingut transferit.

Accedix a una web coneguda (per exemple, *ceice.gva.es*), i comprova la informació enviada i rebuda:



Exercici 2:

Utilitza Google Chrome (opció de Ferramentes per a desenrotlladors, pestanya Network) per a veure l'esquema de petició i resposta HTTP cap a alguna web coneguda. Identifica el codi d'estat, les capçaleres de resposta (tracta d'identificar algunes d'elles) i el contingut.

5. Sistema de noms de domini o DNS

Quan dos equips es comuniquen a través d'Internet, el dispositiu d'origen necessita conèixer dos elements fonamentals de la destinació: la seua **adreça IP**, que identifica de manera única a un equip dins de la xarxa, i el **port**, que indica el servei específic al qual es desitja accedir dins d'eixe equip, com pot ser una pàgina web o un servidor de correu.

Per exemple, en accedir a una pàgina com *www.google.es*, el navegador necessita saber quina és l'adreça IP del servidor on està allotjat el lloc, així com el port del servei web corresponent, que normalment és el **port 80 per a connexions HTTP** o el **443 per a HTTPS**.

Les dos dades són essencials per a establir la connexió correctament. Si no es coneix l'adreça IP, les dades no poden arribar a la destinació; si no es coneix el port, el servidor no sabria quina aplicació ha de respondre a la petició.

Encara que és tècnicament possible escriure directament una adreça IP en el navegador per a accedir a un lloc web, això no és pràctic ni intuïtiu. Les adreces IP són difícils de memoritzar i, a més, poden canviar amb el

temps. Per contra, els noms de domini com *www.google.es* són més fàcils de recordar i comprendre. No obstant això, atès que els equips de xarxa funcionen internament amb adreces IP, és necessari un sistema que realitzi la traducció entre noms i adreces.

Eixe sistema és el **DNS**, o **Sistema de Noms de Domini** (*Domain Name System*), que s'encarrega de traduir noms de domini a adreces IP i viceversa. Este procés es denomina **resolució DNS** i resulta fonamental per al funcionament d'Internet.

El DNS utilitza una base de dades que presenta una estructura jeràrquica i distribuïda.

- És **jeràrquica** perquè organitza els noms de domini en diversos nivells successius, des de l'arrel fins als subdominis (per exemple: *.és* → *.gob.es* → *educacion.gob.es*).
- És **distribuïda** perquè la informació no està emmagatzemada en un únic servidor central, sinó repartida entre milers de servidors DNS a tot el món, el que proporciona redundància, escalabilitat i disponibilitat del sistema.

El DNS admet tant direccions **IPv4** com **IPv6**, i emmagatzema la informació necessària en registres especials anomenats **registres de recursos** (*Resource Records* o *RR*). Estos registres poden contindre dades com a adreces IP, àlies de dominis, informació de servidors de correu, entre altres. Estos registres s'agrupen en zones, que són parts de l'espai de noms gestionades per un servidor autoritatiu responsable de mantindre i proporcionar eixa informació de forma actualitzada.

A continuació s'aprofundirà en estos elements i en el funcionament detallat del sistema DNS.

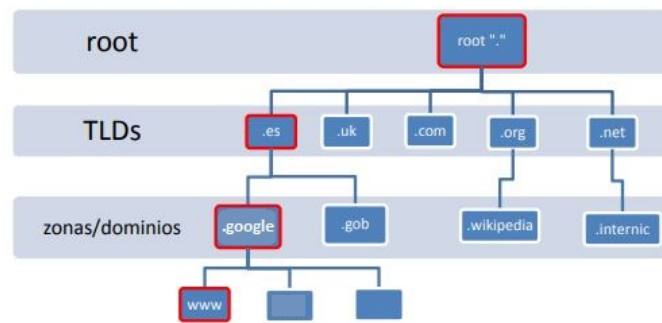
5.1. Elements integrants del DNS

El DNS s'estructura en tres components principals: el **espai de noms de domini**, els **servidors de noms** i els **resolvers**.

5.1.1 Espai de noms de domini

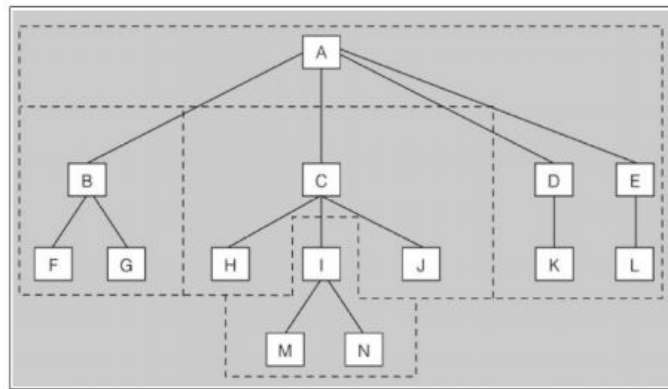
L'espai de noms de domini és una estructura jeràrquica en forma d'arbre invertit on cada node conté **registres de recursos** (*RR*) que proporcionen informació sobre el component maquinari i programari que recolzen este domini, com per exemple, els hosts, els servidors de noms, els servidors web, els servidors de correu, etc.

Del node arrel, situat en el nivell més alt, partixen les branques que conformen les zones. Estes, al seu torn, poden contindre un o més nodes o dominis que al seu torn poden dividir-se en subdominis segons es baixa en la jerarquia.



- Cada node de l'arbre es diu domini i rep una etiqueta o nom. El nom de domini d'un node es crea mitjançant la concatenació de totes les etiquetes, començant per este node i acabant en el node arrel. Per tant, tot domini complet acaba en un punt final "." que indica el final de l'espai en la zona arrel.
- El nivell més alt de tota la jerarquia és el **domini arrel** o **root**, i es representa per "." (punt). En el sistema DNS un node pot tindre un nom de fins a 63 caràcters. La profunditat de nodes està limitada a 127 nivells.
- Just un nivell per davall es troben els **Top Level Domains** o **TLDs**, els quals estan allotjats en el que es coneix com a **servidors arrel**. Existixen diversos servidors arrel situats en diferents continents. S'identifiquen amb les lletres de l'alfabet, i diversos d'ells es troben dividits físicament i dispersos geogràficament amb la finalitat d'augmentar el seu rendiment. Els seus noms són de la forma *lletra.root-servers.org*. Són gestionats per organitzacions independents. Per exemple, *a.root-servers.org* és gestionat per VeriSign, Inc. Pots trobar més informació sobre estos servidors en <https://root-servers.org/>.
- Respecte als TLDs, existixen 3 tipus:
 - **Dominis de nivell superior geogràfics (ccTLD)**: són aquells TLDs que representen a països. Utilitzen els codis de país de dos lletres definits en estàndard ISO-31663. Per exemple, *.és* per a Espanya, *.fr* per a França, *.pt* per a Portugal, etc.
 - **Dominis de nivell superior genèrics (gTLD)**: aquells TLD que estan oberts a qualsevol persona o organització. Estan formats per tres o més caràcters. Per exemple, *.biz* per a negocis, *.com* per a propòsits comercials, *.info* per a informació general, etc.
 - **Dominis de nivell superior d'infraestructura (uTLD)**: esta categoria té un únic TLD, *.arpa*, que s'utilitza exclusivament per a la gestió de la infraestructura.
- Per davall dels TLDs apareixen els **dominis de segon nivell**, com *gob.es*, i estos al seu torn poden contindre subdominis (*educacion.gob.es*, *www.educacion.gob.es*, etc.).
- La jerarquia continua fins a arribar a un node final que representa un recurs. Per exemple "*www.miweb.com*" realment és "*www.miweb.com.*", on el punt final més a la dreta representa la zona arrel, encara que este últim normalment no es mostra. El nom format per tota la cadena es coneix com **Fully Qualified Domain Name (FQDN)**.

És important no confondre els conceptes de **zona** i **domini**. Mentre el domini fa referència al nom lògic en la jerarquia del DNS, una zona és la porció de la base de dades que administra un servidor DNS determinat. Un domini pot dividir-se en diverses zones si es distribueix la seua gestió.



En la figura anterior hi ha tants dominis com requadres de lletres agrupats en 4 zones delimitades per les línies discontinuades. El nom de domini corresponent a cada zona (es nomenen segons el seu node superior) és A, B.A, C.A, I.C.A. Cadascuna d'aquestes quatre zones tindrà un o més servidors DNS per a gestionar-la.

5.1.2 Servidors de noms

Els servidors de noms són servidors encarregats de mantindre i proporcionar informació de l'espai de noms o dominis.

- **Servidors autoritatius:** emmagatzemen informació completa per a una o diverses zones de les quals són responsables. Proporcionen respostes sobre els dominis per als quals han sigut configurats. L'estàndard que definix el DNS estableix que tota zona ha de tindre, almenys, dos servidors autoritatius:
 - El **primari**, que guarda i administra les versions definitives dels registres de recursos de la zona
 - El **secundari**, que guarda una còpia que s'actualitza cada vegada que es produïx un canvi, a través d'un procés conegut com a **transferència de zona**. Això proporciona un mecanisme de redundància, robustesa, rendiment i còpia de seguretat, ja que si el servidor primari falla, la xarxa continua operativa gràcies al servidor secundari.
- **Servidors cau** (o servidors recursius): emmagatzemen temporalment respostes obtingudes d'altres servidors autoritatius. Gràcies al seu cau, reduïxen el nombre de consultes externes i milloren el rendiment de la xarxa.

Amb esta organització de servidors de noms, i la seua intercomunicació, s'aconsegueix la distribució i redundància de l'espai de dominis.

5.1.3 Resolvers

El resoladre és una rutina o component del sistema operatiu que actua com a intermediari entre les aplicacions de l'usuari (com un navegador) i els servidors DNS. Quan una aplicació necessita l'adreça IP corresponent a un nom de domini, el resoladre s'encarrega de gestionar eixa petició i retornar la informació en el format que el sistema puga utilitzar. Normalment, es comunica amb un servidor DNS configurat per defecte en el sistema o la xarxa.

5.2 Procés de resolució DNS

La principal funció d'un servidor DNS és respondre a consultes de clients o d'altres servidors DNS. Existixen dos tipus principals de consultes:

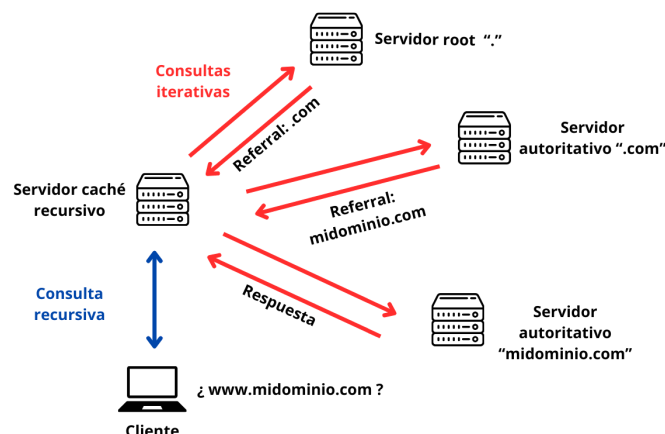
- **Consulta recursiva:**

- El *resolver* (client) demana al seu servidor DNS local una resposta basada en els seus arxius de zona o cau.
- Si el servidor no té la informació sol·licitada, respon amb el punter al servidor arrel ".".
- El *resolver* consulta al servidor arrel, després els servidors de domini de nivell superior (TLD), i així successivament, fins a trobar el servidor autoritatiu que té la resposta correcta.
- Normalment, el *resolver* fa una consulta recursiva a un servidor DNS intermedi que actua com a cau recursiu, evitant que el client haja de realitzar múltiples consultes.

- **Consulta Iterativa:**

- El *resolver* sol·licita al servidor DNS una resposta o un error si el recurs no existix.
- El servidor DNS actua com a intermediari, realitzant consultes iteratives a altres servidors DNS fins a obtenir la resposta o un error.

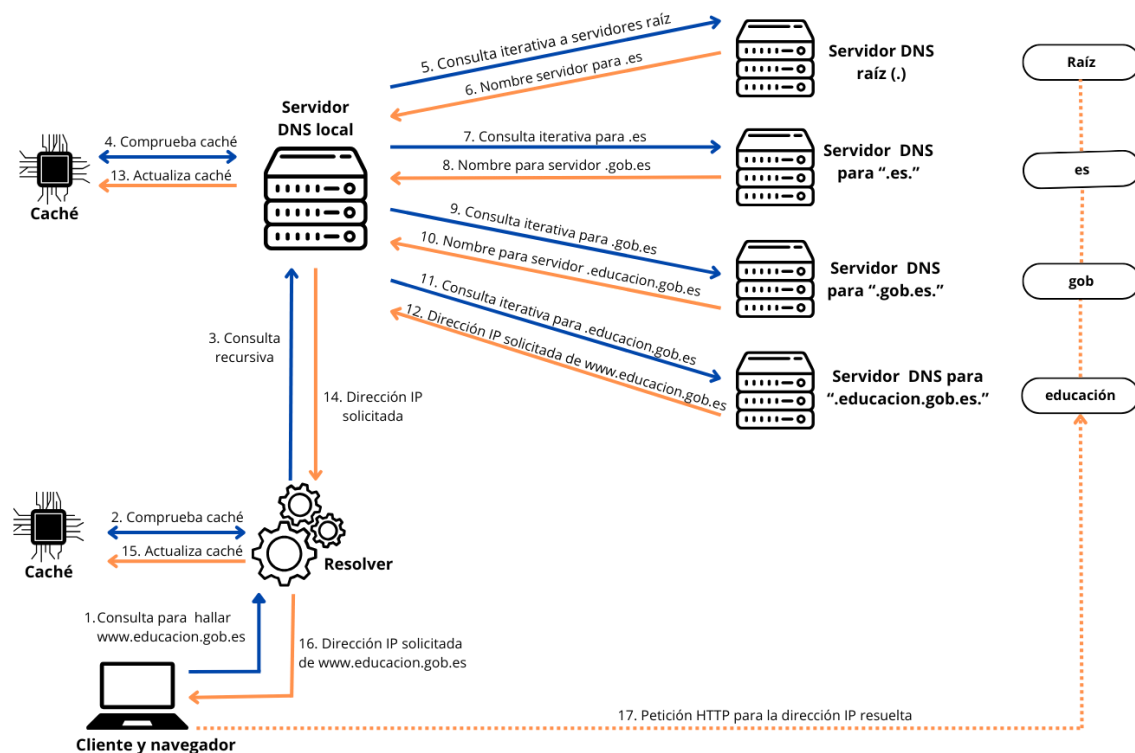
Per tant, les consultes recursives són generalment generades pels clients DNS, mentres que les consultes iteratives són creades pels servidors DNS en consultar a altres servidors.



En termes generals, el procés que se segueix en una resolució DNS és el següent.

- El client (*resolver*) envia la consulta al seu servidor DNS.
- Si el servidor DNS té autoritat sobre el domini consultat, retorna la resposta usant els seus propis registres.
- Si el servidor no té autoritat i no és recursiu, informa al *resolver* on ha de dirigir la seua consulta.
- Si el servidor no és autoritatiu però és recursiu, realitza consultes iteratives per a trobar el servidor autoritatiu i retorna la resposta al client, emmagatzemant la informació en cau per a futures consultes.

Vegem un exemple concret de resolució de noms per a comprendre millor el procés. Suposem que un client necessita localitzar l'equip `www.educacion.gob.es` i envia una petició al seu servidor DNS. Assumim també que el seu servidor no té informació sobre eixe domini. Així, observarem com es duu a terme la resolució completa



- Consulta inicial del client:** un client, mitjançant un navegador web, sol·licita la resolució del nom de domini `www.educacion.gob.es`. El *resolver* consulta la seua cau per a veure si té l'adreça IP emmagatzemada.
- Consulta al servidor DNS local:** si l'adreça no està en la cau, el *resolver* envia una consulta recursiva al servidor DNS primari o local configurat. Este servidor pot estar dins de l'empresa o en Internet. El servidor DNS local revisa la seua cau i, si troba la informació, retorna l'adreça IP corresponent.
- Consultes iteratives del servidor DNS:** si el servidor DNS local no pot resoldre la consulta, inicia una sèrie de consultes iteratives per a trobar el servidor autoritatiu del domini:
 - Servidor arrel:** consulta al servidor arrel. Si no és autoritatiu, el servidor arrel retorna el nom del servidor responsable del domini de primer nivell (.és).
 - Servidor TLD:** consulta al servidor del TLD (.és). Si no és autoritatiu, retorna el nom del servidor responsable del domini de segon nivell (.gob.es).
 - Servidor del domini específic:** consulta al servidor del domini de segon nivell (.gob.es). Si no és autoritatiu, retorna el nom del servidor responsable del domini específic (`educacion.gob.es`).
- Resposta del servidor autoritatiu:** finalment, s'arriba al servidor autoritatiu per al domini `educacion.gob.es`, que retorna l'adreça IP de `www.educacion.gob.es` al servidor DNS local.
- Actualització i resposta:**
 - El servidor DNS local actualitza la seua cau amb la nova informació.
 - El servidor DNS local envia la resposta al *resolver* del client.
 - El *resolver* del client actualitza la seua cau.
 - La resposta arriba al navegador web en el format adequat, completant la sol·licitud de resolució.

Este procés consumix bastant amplada de banda, per la qual cosa és recomanable implementar una solució DNS interna amb cau, especialment en xarxes amb alt trànsit cap a l'exterior, com a col·legis, universitats, biblioteques o grans empreses. Això permet obtenir temps de resposta més ràpids i una millor experiència de navegació web.

Eina DIG

DIG (*Domain Information Gopher*) és una eina de línia de comandes que realitza cerques en els registres DNS, a través dels noms de servidors, i mostra el resultat. Es pot executar tant en Linux com en Windows encara que, en aquest últim, és possible que siga necessària la seua instal·lació prèvia.

Vegem un exemple de com funciona amb la comanda `dig www.gva.es`:

```

Simbolo del sistema
Microsoft Windows [Versión 10.0.18362.1016]
(c) 2019 Microsoft Corporation. Todos los derechos reservados.

C:\Users\May>dig www.gva.es

;<<>> DiG 9.16.5 <<>> www.gva.es
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 8975
;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 1460
;; QUESTION SECTION:
;www.gva.es.                IN      A

;; ANSWER SECTION:
www.gva.es.                 70939   IN      CNAME   simac13.gva.es.
simac13.gva.es.             5       IN      A       193.144.127.85
simac13.gva.es.             5       IN      A       195.77.16.26

;; Query time: 16 msec
;; SERVER: 80.58.61.254#53(80.58.61.254)
;; WHEN: Sat Aug 15 14:36:15 Hora de verano romance 2020
;; MSG SIZE rcvd: 93

```

Explicació de l'eixida:

- Totes les línies que inicien amb ";" són comentaris.
- La primera línia ens retorna la versió del comando dig que estem utilitzant.
- Després mostra una **secció de consulta** (*question section*) amb la nostra consulta.
- A continuació, apareix la **secció de resposta** (*answer section*) que conté la resposta del servidor DNS.
 - Esta secció pot incloure diferents tipus de registres de recurs (RR). Per defecte, la resposta està associada al registre de recurs de direcció A (RR A) i al registre de recurs de nom canònic (RR CNAME), però també es poden consultar altres registres.
 - El **registre de direcció A** (RR A) associa noms de domini (FQDN) a direccions IPv4. El registre A té l'estructura següent: *NomDomini IN A IP*.
 - El **registre de recurs nom canònic** (RR CNAME) enllaça el nom de domini amb un àlies, és a dir, amb un altre nom que també redirigix al mateix contingut. El verdader nom, per tant, és el que es connecta a través del registre A amb l'adreça IP. L'avantatge d'este sistema és que, si l'adreça IP canvia, només és necessari modificar el registre A. Com que tots els àlies es guien segons este registre A. La seua estructura és la següent: *NomDomini IN CNAME Nom canònic o IP*
- **Interpretació de la resposta:**

- Primer es mostra el registre CNAME que indica que *www.gva.es* és un àlies de *simac13.gva.es*.
- Després es mostren dos registres A per a *simac13.gva.es*, proporcionant les seues adreces IP.
- Cada registre comença amb el nom complet del domini (incloent el punt final), seguit del temps en segons que romandrà el registre en la memòria cau.

- **Estadístiques de la consulta:**

- Finalment, es mostren estadístiques com el temps de consulta, el servidor que va respondre, la data i hora de la consulta i la grandària del missatge rebut.

Si s'executa DIG especificant l'opció **+trace**, es mostra una traça dels servidors pels quals passa la consulta fins a arribar al servidor autoritatiu, permetent veure la llista completa de nodes i passos de resolució d'un nom. Això és molt útil per a entendre com funciona el sistema de noms de domini.

6. Patrons de disseny de programari

Després de revisar els tipus d'aplicacions web, l'arquitectura bàsica, els protocols més comuns i el paper del DNS, arribem a un component essencial en el desenvolupament d'aplicacions web robustes, mantenibles i eficients: els patrons de disseny.

Un **patró de disseny** és un conjunt de bones pràctiques, directrius estructurals i components organitzats que proporcionen una solució reutilitzable a problemes comuns en el desenvolupament de programari. El seu objectiu principal és estandarditzar la forma en què es dissenyen i construeixen les aplicacions, facilitant la comprensió del sistema, la col·laboració entre desenrotlladors i la reutilització de components. A més, permeten generar estructures modulars, predictibles i escalables, fent que el codi siga més fàcil de mantindre i estendre.

En l'àmbit de les aplicacions web, existixen diversos patrons de disseny que ens orienten a l'hora d'organitzar el codi, separar responsabilitats i estructurar els distints components. Un dels més utilitzats (i probablement el més conegut) és el patró **MVC**, que veurem a continuació. A partir d'ell han sorgit altres enfocaments, dissenyats per a adaptar-se a necessitats concretes, entorns moderns o filosofies distintes d'interacció. En este apartat revisarem MVC i algunes de les seues variants més rellevants.

6.1. El patró MVC

MVC són les sigles de **Model-Vista-Controlador** (*Model-View-Controller*), i representa un dels patrons més utilitzats tant en aplicacions web com en moltes aplicacions d'escriptori.

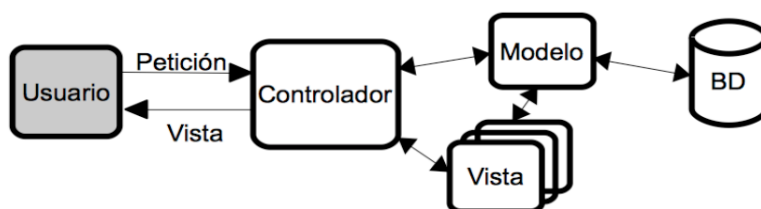
Este patró es basa en la divisió del sistema en tres components fonamentals, cada un amb una responsabilitat clarament definida:

- El **model** representa les dades i la lògica de negoci de l'aplicació. Gestiona l'accés, emmagatzematge i validació de la informació, normalment mitjançant estructures, classes o objectes que interactuen amb bases de dades o altres sistemes d'emmagatzematge. El model no coneix ni depèn dels altres components, la qual cosa afavorix la reutilització i l'aïllament de la lògica principal.

- La **vista** és la part de l'aplicació que s'encarrega de mostrar la informació a l'usuari i d'arreglar la seua interacció. Està composta per pàgines, formularis, interfícies gràfiques o altres elements visuals. La seua única funció és presentar les dades i arreglar esdeveniments (clics, formularis, etc.), sense contindre lògica de negoci.
- El **controlador** actua com a intermediari entre el model i la vista. Arregla les peticions de l'usuari (per exemple, mitjançant una URL o un formulari), processa la lògica necessària, accedix al model per a recuperar o modificar dades, i determina quina vista ha de mostrar-se com a resposta. Pot haver-hi múltiples controladors, segons les funcionalitats de l'aplicació.

Una dels principals avantatges d'este patró és que separa clarament responsabilitats, la qual cosa permet dividir el treball entre diferents perfils professionals. Per exemple, un dissenyador pot centrar-se en les vistes sense necessitat de conèixer la lògica del model o del servidor, mentre que un desenrotllador *backend pot treballar en el controlador i el model sense preocupar-se del disseny visual.

En forma d'esquema, podríem representar-ho així:



6.2. Altres alternatives a MVC

A partir de la popularitat i estructura de MVC han sorgit altres patrons que ho adapten o reinterpreten per a entorns específics. En la majoria d'estos patrons es manté el model de dades i la vista com a components clau, i es modifica o reemplaça el paper del controlador. Esta família de patrons es coneix genèricament com **MVW**, (*Model-Vista-Qualsevol cosa*, o *Model-View-Whatever*), la qual cosa reflectix la flexibilitat del tercer component.

Estos patrons solen buscar una major modularitat, eliminar la lògica centralitzada dels controladors o facilitar la interactivitat directa entre model i vista. A continuació, es descriuen els més representatius.

El patró MVVM

El patró **MVVM** (*Model-Vista-VistaModel*, o *Model-View-ViewModel*) substitueix el controlador de MVC per un nou component denominat **ViewModel** (VistaModel), i se centra exclusivament en els components del model i la vista.

En este enfocament, l'usuari interactua directament amb la vista, que està vinculada al model a través del ViewModel. Este component intermedi conté la lògica necessària per a transformar les dades del model en una forma adequada per a ser mostrada en la vista, i també arregla les accions de l'usuari per a modificar el model. La comunicació entre el model i la vista és bidireccional: els canvis en la vista actualitzen el model, i els canvis en el model es reflectixen automàticament en la vista. Això es coneix com **data binding** (enllaç de dades).

MVVM és especialment útil en aplicacions on hi ha una única vista principal que s'actualitza dinàmicament, com és el cas de moltes **Single Page Applications** (SPA), on no es recarreguen pàgines completes sinó que s'actualitzen seccions específiques. Atés que en estes aplicacions no és necessari decidir quina vista carregar (només hi ha una), el controlador perd rellevància i pot ser substituït per este vincle directe model-vista. Frameworks moderns com *Angular* o **Vue.*js* incorporen conceptes de MVVM, amb diferents nivells de formalitat.

El patró MOVE

El patró **MOVE** (*Model-Operació-Vista-Esdeveniment*, o *Model-Operation-View-Event*) proposa una divisió distinta del rol del controlador, reemplaçant-lo per dos components: **esdeveniments** i **operacions**. Els esdeveniments representen accions o successos que ocorren en l'aplicació, generalment generats per la interacció de l'usuari (com fer clic en un botó, enviar un formulari, etc.). Estos esdeveniments desencadenen operacions, que definixen les tasques que l'aplicació pot realitzar (com validar dades, consultar el model o actualitzar la interfície).

En este patró, quan ocorre un esdeveniment, s'executa una operació associada, que pot accedir al model per a obtindre o modificar dades, i generar una vista que es mostra a l'usuari com a resposta. D'esta manera, la lògica que en MVC estava concentrada en el controlador es repartix entre els esdeveniments que l'activen i les operacions que la implementen. Esta separació pot facilitar l'organització del codi i el maneig d'aplicacions amb moltes accions simultànies o asincròniques.

El patró MVP

El patró **MVP** (*Model-Vista-Presentador*, o *Model-View-Presenter*) substituïx el controlador de MVC per un component anomenat **presentador**. A diferència del controlador, que en MVC pot gestionar múltiples vistes, en MVP cada vista té associat el seu propi presentador, la qual cosa reforça el *encapsulamiento

En este enfocament, la vista és passiva i no interactua directament amb el model. Tot el flux passa pel presentador: quan l'usuari realitza una acció, esta s'envia al presentador, que accedix al model, obté o actualitza dades, i després actualitza la vista. Això crea una relació clara entre cada parell vista-presentador, la qual cosa facilita la separació de responsabilitats i millora la capacitat de realitzar proves automatitzades.

El patró MVP és molt comú en el desenvolupament d'interfícies gràfiques complexes, tant en aplicacions d'escriptori com mòbils, com en entorns .NET o aplicacions Android.

7. Recursos necessaris

Per a implantar una aplicació web i que els clients puguin utilitzar-la, és necessari comptar amb una sèrie de recursos tant de maquinari com de programari.

- En el costat del **client**, simplement caldrà comptar amb un equip amb el maquinari necessari (depenent de l'aplicació web que siga, podrà ser un mòbil, una tauleta, portàtil, PC...), i, típicament, un navegador web instal·lat (encara que també podria tractar-se d'una aplicació híbrida per a mòbil, o d'escriptori, i en este cas faria falta l'aplicació en si).

- En el costat del **servidor**, normalment necessitem almenys un equip amb característiques de maquinari adequades, especialment pel que fa a processador, memòria RAM i capacitat d'emmagatzematge. En eixe servidor serà necessari instal·lar:
 - Un **servidor web** o **servidor d'aplicacions**, encarregat d'allotjar l'aplicació web i atendre les peticions dels clients. És habitual comptar amb dos entorns diferenciats: un servidor de producció, accessible des d'Internet i que conté la versió final de l'aplicació, i un servidor de proves o desenvolupament, situat localment per a fer les tasques de programació, proves i manteniment abans de publicar els canvis.
 - Un **servidor de base de dades** (si l'aplicació el requereix), o un **sistema gestor de bases de dades** (SGBD). Este component és fonamental per a la majoria d'aplicacions web, ja que moltes d'elles treballen amb dades emmagatzemades i requereixen consultar, inserir o modificar informació de manera persistent.

7.1. Llenguatges

En el desenvolupament d'aplicacions web és important identificar els llenguatges de programació utilitzats, que es classifiquen segons l'entorn en què s'executen:

- **Llenguatges de l'entorn client** (o llenguatges client): s'executen en el navegador de l'usuari i permeten la interacció directa amb l'aplicació. En este context, s'utilitzen llenguatges com a **HTML** i **CSS** per al disseny i estructura de les pàgines (encara que no es consideren llenguatges de programació en sentit estricte), i **JavaScript** per a manejar esdeveniments, validar formularis, crear animacions, etc. També existixen ferramentes i preprocessadors com **SASS**, que amplien les capacitats de CSS, així com **frameworks i llibreries JavaScript** com a **Angular**, **React** o **Vue**, que faciliten el desenvolupament d'interfícies dinàmiques. A més, existixen llenguatges com **TypeScript**, una evolució de JavaScript que afig tipat estàtic i major estructura, la qual cosa facilita el desenvolupament d'aplicacions grans i escalables.
- **Llenguatges de l'entorn servidor** (o llenguatges servidor): s'executen en el servidor web i permeten realitzar operacions com a consultes a bases de dades, maneig d'arxius o gestió de sessions. Un dels llenguatges més utilitzats tradicionalment en este àmbit ha sigut **PHP**, que continua sent molt comú per a aplicacions web dinàmiques, especialment en combinació amb servidors Apache. Un altre llenguatge àmpliament adoptat és **JavaScript**, que gràcies a l'entorn d'execució **Node.js** també pot utilitzar-se en el servidor, permetent escriure tant el backend com el frontend d'una aplicació en un únic llenguatge. Sobre Node.js es dona suport a l'ús de **TypeScript**, que s'està consolidant com una ferramenta clau en el desenvolupament modern del costat servidor, en aportar tipat estàtic, una major organització del codi i millor suport per a projectes de gran grandària. En aplicacions que utilitzen tecnologia Microsoft, també s'empra **ASP.NET**, que s'executa sobre el servidor IIS i s'integra amb l'ecosistema Windows.

7.2. Exemples de programari

A continuació es descriuen els programes concrets que s'utilitzaran durant el curs, tant en el costat client com en el servidor.

En el **client**, necessitarem un **navegador web** per a visualitzar i interactuar amb les aplicacions. Existixen diverses opcions disponibles, com Mozilla Firefox, Google Chrome, Microsoft Edge (abans Internet Explorer), Safari o Opera, encara que probablement les millors alternatives per compatibilitat, rendiment i eines per a desenvolupadors són Google Chrome i Mozilla Firefox.

En el **servidor**, el programari a utilitzar dependrà del llenguatge de programació triat i del sistema operatiu, encara que en el nostre cas, utilitzarem entorns compatibles amb Windows.

- Per al desenvolupament amb **PHP**, emprarem el servidor web Apatxe, instal·lat mitjançant eines com XAMPP, que integren Apatxe, el llenguatge PHP i el sistema gestor de bases de dades MySQL/MariaDB en un únic paquet de fàcil configuració.
- Per a desenvolupament amb **JavaScript** del costat servidor, s'utilitzarà **Node.js**, acompanyat de diversos mòduls i ferramentes que s'aniran introduint al llarg del curs. En este entorn també s'instal·larà el framework Express, que facilita la creació i organització de rutes, així com la gestió de les peticions HTTP.

Respecte al **servidor de bases de dades**, existixen múltiples alternatives. D'una banda, tenim les **bases de dades relacionals**, com MySQL/MariaDB, PostgreSQL, Oracle o SQL Server, totes elles disponibles en versions gratuïtes o comercials segons les necessitats del projecte. D'altra banda, existixen opcions **no relacionals o NoSQL*, com MongoDB, que permeten emmagatzemar documents en format JSON i s'adapten especialment bé a estructures de dades flexibles i dinàmiques.

En el nostre curs, utilitzarem **MySQL/MariaDB** (integrat amb Apatxe i PHP mitjançant XAMPP) i **MongoDB** (en combinació amb Node.js) com a solucions principals per a emmagatzematge de dades.

IDEs

Per al desenvolupament d'aplicacions és necessari comptar amb un bon entorn de desenvolupament o IDE amb el qual editar el codi, depurar i provar les aplicacions.

Existeixen multitud d'opcions disponibles, la majoria d'elles gratuïtes. Des d'entorns de propòsit general vàlids per a molts llenguatges (Atom, Sublim, Visual Studio Code...) a uns altres més específics i orientats a algun llenguatge en concret, com PhpStorm.

En el nostre cas, optarem per **Visual Studio Code**, per la seua versatilitat i popularitat creixent. Permet instal·lar multitud d'extensions per a treballar amb diferents tipus de llenguatges i frameworks, i ens permet també una fàcil integració amb PHP i Node.js.

7.3. Webs d'interés

A continuació s'enumeren algunes webs on consultar o descarregar els recursos indicats en aquest tema, per al desenvolupament d'aplicacions en entorn servidor.

- XAMPP - <https://www.apachefriends.org/es/index.html>
- Apache - <https://www.apache.org>
- Node.js - <https://nodejs.org>
- MongoDB - <https://www.mongodb.com>

- Express - <https://expressjs.com>
- Laravel - <https://laravel.com>
- Symfony - <https://symfony.com>

Exercici 3:

Utilitza l'eina [Google Trends](#) per a buscar els termes de *Laravel*, *Symfony* i *Node.js*. Dedueix a partir de les cerques quin d'ells creus que és més popular actualment. Després, acudeix a la web de [InfoJobs](#) i busca ofertes de treball amb aquests tres frameworks, per a determinar quin és el més demandat en l'actualitat.